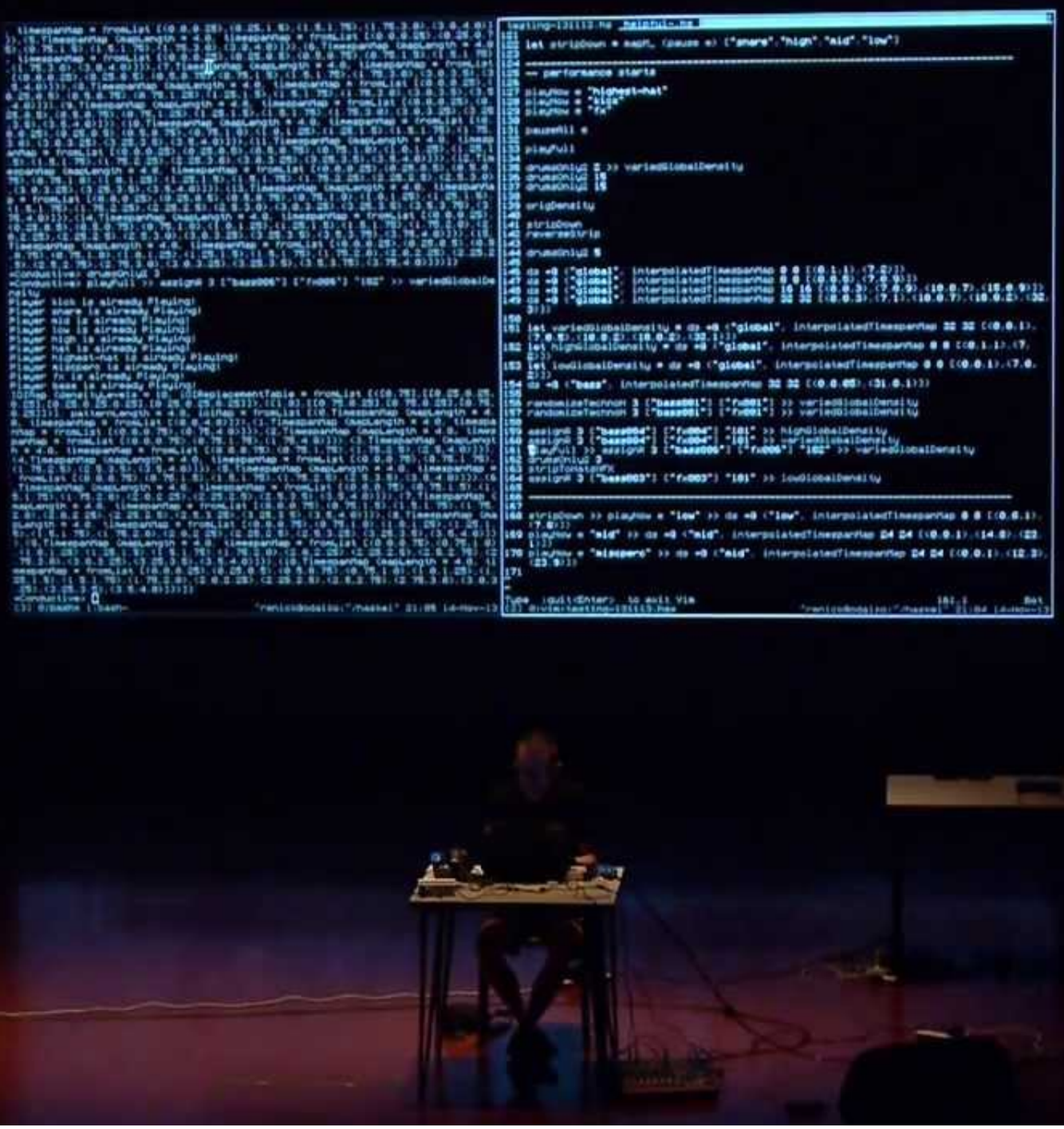
INSTITUTE OF SONOLOGY

Royal Conservatoire The Hague

Live Coding

Composing with Algorithms

http://www.bjarni-gunnarsson.net



Live Coding, A User's Manual

"Live coding has been described in terms such as writing software in real time, changing a program while it is running, projecting the screen for the audience to participate in, writing as an improvisatory practice, composing live using textual notation, changing rules while following them, conversational programming (conversing with the computer in its own native language), thinking in public, and creating and using bespoke systems tailored for on- the- fly or just- in- time performance."

(Live Coding, A User's Manual)

LIVE CODING

_a user's manual

Alan F. Blackwell
Emma Cocker
Geoff Cox
Alex McLean
Thor Magnusson

```
function newPattern () {
 var type = document.querySelectorAll('path'), i;
 currentScale = 1;
 currentTime = setInterval(function() {
  for (i = 0; i < type.length; ++i) {
    type[i].style.transform='scale('+ currentScale +')';
    currentScale = Math.random() * 2;
  }
 }, 2000);
}
newPattern()
```

Live Coding

"Live coding is a new direction in electronic music and video, and is getting somewhere interesting. Live coders expose and rewire the innards of software while it generates improvised music and/or visuals. All code manipulation is projected for your pleasure. Live coding works across musical genres, and has been seen in concert halls, late night jazz bars, as well as algoraves. There is also a strong movement of video-based live coders, writing code to make visuals, and many environments can do both sound and video, creating synaesthetic experiences. Live coding is inclusive and accessible to all."

(From toplap.org)

Live Coding

WIRED BUSINESS CULTURE GEAR IDEAS SCIENCE SECURITY TRANSPORTATION SIGN IN

MICHAEL CALORE CULTURE 03.26.2019 09:00 AM

DJs of the Future Don't Spin Records—They Write Code

"Live-coding" parties are the latest phenomenon in underground electronic music culture.

Underground music in 2017

Run the code: is algorave the future of dance music?

WIRED STAFF SCIENCE 07.03.06 12:00 PM

Real DJs Code Live

LONDON – Some DJs spin vinyl or twiddle fade
subroutines in C++.

A new brand of music maestro is turning progra
performance, eschewing turntables for a compi

Live Coding

Live Coding is a recent movement aimed at ***creating***, ***manipulating*** and ***projecting*** the computer code itself during a performance of computer music or digital audiovisuals.

Algorithms communicate compositional desires, why not project those to our audience ?

With our modern languages for digital arts, why should we not explore the true dynamics offered by modern programming languages instead of falling back to conventional GUI's, controllers and other ideas that provide much less flexibility ?

Live Coding

"Supports a strong emphasis on formal experiments, reductionism, and functionalism."

"*Engages with the audience through various channels, such as sitting among them while performing (as with the band PowerBooks UnPlugged), allowing people to contribute to the coding of dance performers (e.g., Kate Sicchio's work), or submitting code through Twitter (as in my own work with the live-coding environment "ixi lang"). One of the fundamental tenets of the TOPLAP manifesto (available online at toplap.org/wiki/ ManifestoDraft) is "Show us your screens."*

(Thor Magnusson, Computer Music Journal 2014)

Toplap Manifesto

Live coding contains many manifestos, key texts, and custom coding platforms. From the draft of the ***toplap manifesto***:

** Give us access to the performer's mind, to the whole human instrument.*

** Obscurantism is dangerous. Show us your screens.*

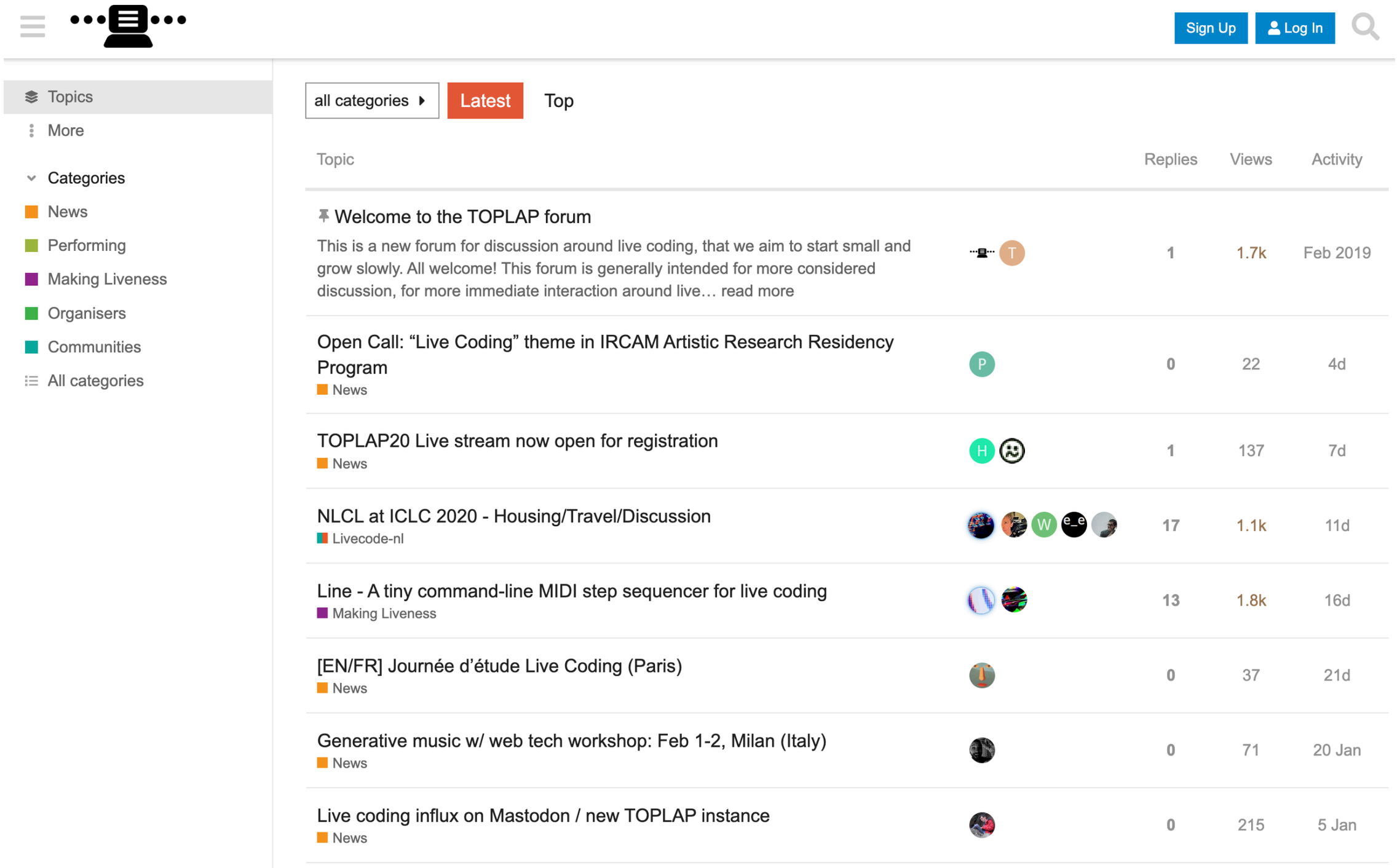
** Programs are instruments that can change themselves.*

** The program is to be transcended - Artificial language is the way.*

** Code should be seen as well as heard, underlying algorithms viewed as well as their visual outcome.*

** Live coding is not about tools. Algorithms are thoughts. Chainsaws are tools. That's why algorithms are sometimes harder to notice than chainsaws.*

Toplap

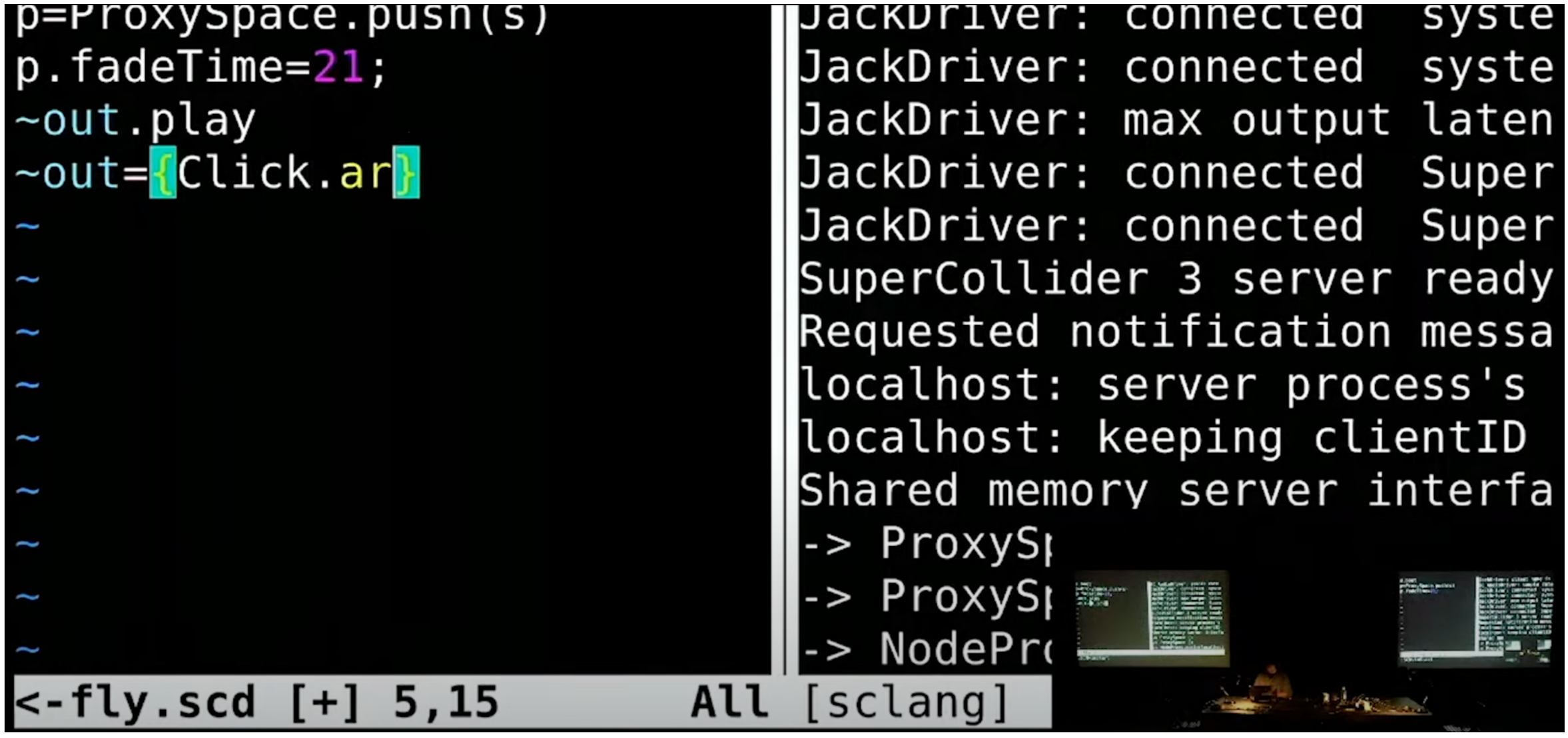


https://toplap.org/

Live Coding

https://www.youtube.com/watch?v=S2EZqikCIfY&t=620s

Live Coding



https://www.youtube.com/watch?v=ntFMuvv2-TY

History (Collins)

- *2000, slub start to play in London projecting their screens*
- *Julian Rohrhuber exploits SuperCollider 2 to allow hotswapping code in performance*
- *2000: ICMC Berlin, networked code passing McCartney/ Rohrhuber*
- *2002: First live coding albums (unreleased)*
- *2003: ChucK*
- *2004: TOPLAP*
- *2005 TOPLAP transmediale*
- *2005: first live code battle*
- *2005: fluxus*
- *2006: aa cell practice*
- *2007: LOSS live code festival*
- *2009: BBC documentary on pub code*
- *2012: Live Notation AHRC: live arts and live coding* ¤
- *2013: Live Coding festival in Karlsruhe*
- …

Live Coding Areas

Thor Magnusson has defined the following main areas of the practice.

* Languages for algorithmic thinking
* Comprehensibility and intuitiveness of code
* Audience engagement and involvement
* Graphical visualizations of algorithms
* Virtuosity and the system's provision for speed coding
* Compositional expressivenes
* Direct access to the artistic material (e.g. synthesis or temporal patterns)
* Collaboration through network protocol
* "Liveness" and the ability to control existing structures
* Embodiment and physicality
* Live-coding systems as musical pieces or works of art

Composing with Algorithms

Julian Rohrhuber reports live coding to concern:

- "An ***effective performance technique***, live coding addresses the problems of improvisation and on-the-fly decision-making in live performance with machines."
- "The fact that the machine can be ***redefined in real time*** opens up new avenues in compositional and musical performance practices, and, indeed, it seems to be a logical and necessary step in the evolution of human–machine communication"
- "The fact that the machine can be redefined in real time opens up new avenues in compositional and musical performance practices, and, indeed, it seems to be ***a logical and necessary step in the evolution of human–machine communication***."
- "Why not explore the ***interfacing of the language*** itself as a novel performance tool, in stark contrast to pretty but conventional user interfaces?"

Pros and Cons

Table. Pros and cons of live coding performance.

Pros and potential	Cons and dangers
Flexibility; the next section can be anything	You forget the current audio or just take too long while you prepare the next section
A great intellectual challenge	Your concentration halves with the adrenalin/stress/beer of a gig
Arbitrarily complex changes of structure at performance time	It's risky to just run code! No debugging or testing is available
A new form of improvisation	You must accept some failures and ugly moments of sound; there is a trade-off between preparation and gig specificity
Normal performance programs like Reason look dull if the screen is projected – but the arcane text coding systems have allure	Deliberate obfuscation is no great criteria for art!
It's rewarding to see real efforts translated into sound	You apply lots of effort for little payback; you must have back-up code in case inspiration is not forthcoming
Computer languages are immensely rich infinite grammars	Typing is hardly the most visually exciting interfacing method – you'll be bent over a screen for the night unless you code in further external controllers

Collins, McLean, Rohrhuber and Ward

Live Coding

There could be ***weak*** and ***strong*** live coding where weak manipulates already existing code and strong writes from scratch.

"Because of the danger of running very complicated new code live, in practice most composers would content themselves with modifying pre-tested snip- pets. They'd go to a gig with a library of prefabricated and tested code. Certain functional aspects of their engine will already be in place; for instance, some sort of time server or scheduler, and shortcuts for running, managing and killing sound agents. This is not to say that in advanced code one cannot make a massive change to the sound synthesis structure just by changing a few characters."

(***Julian Rohrhuber***)

Live Coding

"Watching the articulations of a live guitarist may enhance the experience of a listener who does not play a musical instrument themselves. Can a live coder elucidate the more abstract thinking gestures of their practice?"

"Generally, a programmer cannot work with their eyes closed; [..]"

(Collins, McLean, Griffiths and Wiggins)

"Art of re-programming; changing your mind about a process once established"

(Nick Collins)

Alexandra Cardenas

"Live coders write, rewrite, and manipulate in real time a system that creates music. Still, live coders are the composers of the music. The software becomes a composition and a piece of art itself.."

https://cargocollective.com/tiemposdelruido/Alexandra-Cardenas

Renick Bell

"Live coding allows us to perform with symbol rather than gesture. The ability to use abstraction in performance frees us from the need to gesture but can also allow us to tie complex behaviors to simple gestures. It brings performance much closer to language and therefore all of the advantages that using language gives us."

http://www.renickbell.net/

Shelly Knotts

"Network music systems with the ability to collect data and provide text based communication in-performance, as well as the possibly controversial ability to restrict access to technical resources, allows us to investigate consensus in the context of musical interaction. Obvious arising questions which are actively explored through the design process include, how can we determine musical consensus and is consensus always desirable."

https://shellyknotts.wordpress.com/about/

Algorithm

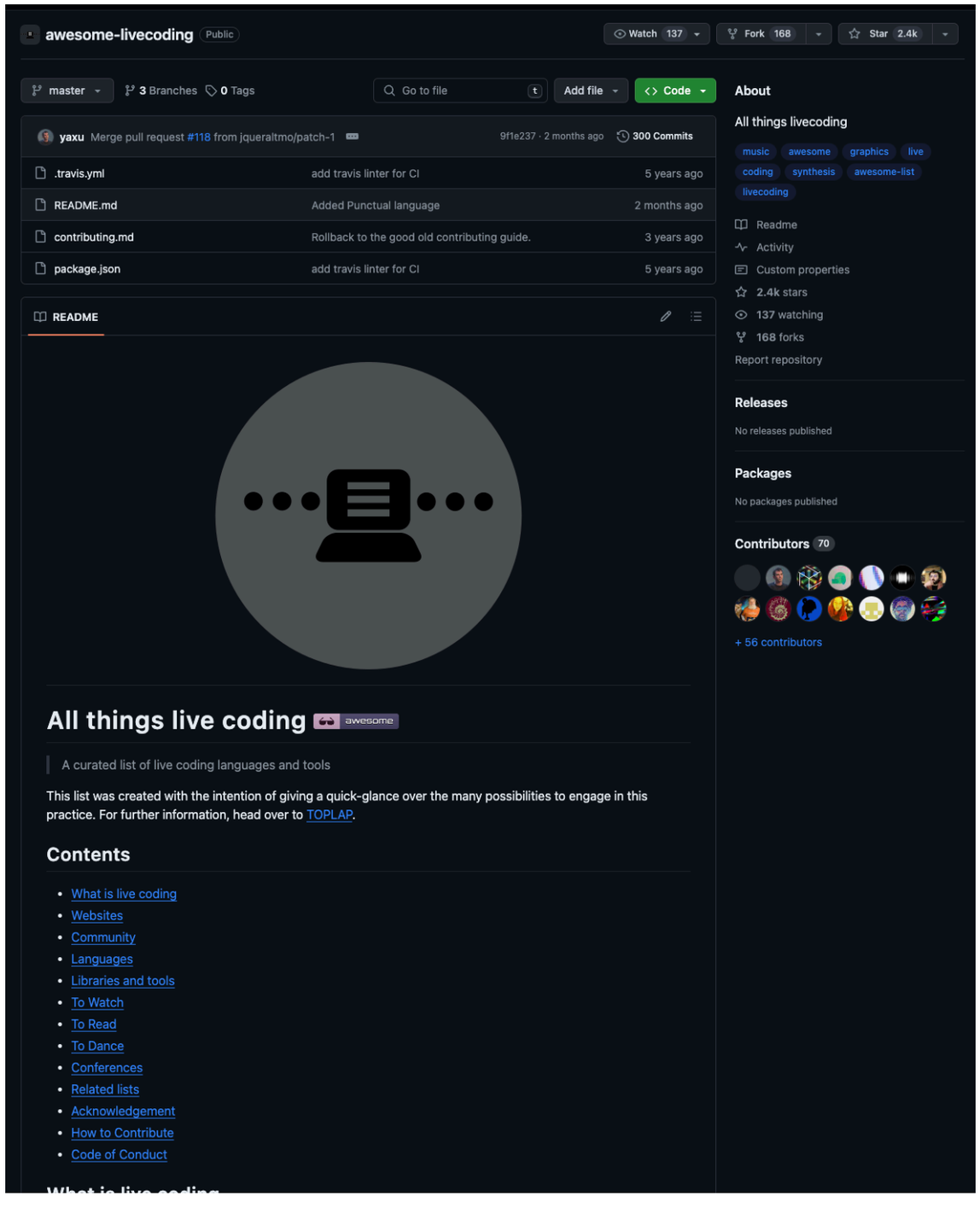
"It is noticeable that in various forms of algorithmic techniques, a peculiar intermediacy between mental and physical acts and a strong affinity toward a reflective attitude by means of mechanisms can be observed. From one point of view, their mechanisms may be seen as an externalization of reasoning. From the other, one can consider them as acting patterns setting cognitive constraints. Addressing the frontier between thinking and acting, the algorithm is necessarily an ambiguous term."

*(**Algorithms in Anthropology. Julian Rohrhuber**)*

Resources

- https://algorave.com/
- https://github.com/toplap/awesome-livecoding
- https://netherlands-coding-live.github.io/
- https://toplap.org/
- Eulerroom

Live Coding



https://github.com/toplap/awesome-livecoding/

ICLC 2023

19 - 23 APRIL IN UTRECHT, THE NETHERLANDS

https://iclc.toplap.org/2023/

Sonology

sonology.org

JITLib

JITLib

Just in time programming is a working methods that includes the programming process itself in the program's operation.

The written code is not aimed at producing a finished tool but rather to be a ***dynamic process*** and ***mode of interaction*** between the programming language and the composer.

JJITLib provides various methods for dynamic modification of a SuperCollider program mainly using ***proxies*** that act as placeholders for objects.

ProxySpace

An attempt to remove any difference between the code that initializes and the code that ***rewrites***.

A ***proxy*** is a place holder that is often used to operate on something that does not yet exist.

Using the ProxySpace the Interpreter and task ***run at the same time*** so we can change all data required by the running process while it runs.

One can replace the current environment with this special type of environment, a ***ProxySpace***. This environment represents processes that play audio on a server.

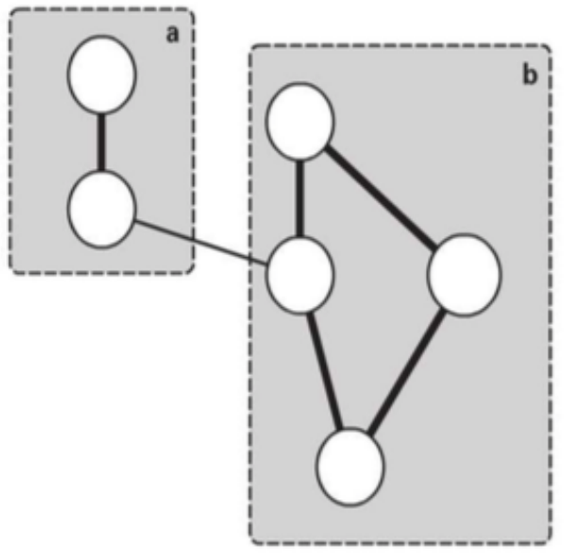
ProxySpace

ProxySpace ***push*** creates such an environment as well as making it active. ProxySpace ***pop*** will free this environment and its associated key, values and environment variables. There is also ***ProxySpace.new*** which would create but not activate the environment. To replace the one created with .new we can use ***currentEnvironment = p***

The difference between ***ProxySpace*** and ***Environment*** is that while Environment contains language object, ProxySpace contains descriptions and ***running instances of synthesis processes*** on the server.

Assigning environment variables in a ProxySpace converts those to ***server nodes*** in a ***synthesis graph*** ready to be used within a sound process and be referenced by other nodes.

JITLib



From the SuperCollider book, chapter 7,
live coding by Rohrhuber and De Campo.

Node Proxy

Audio output on the server has two main properties:
* A calculation ***rate*** (audio or control)
* A certain ***number of channels***.

These are the main static properties of a node proxy, which cannot be changed while it is in use.

UGen graphs always have a ***fixed rate*** and ***number of channels*** and this can either be set implicitly or as a result of what the node initially contains. A neutral proxy (x.play) will be initialized to two output channels. If we need to reset a node proxy the ***.clear*** message can be sent.

To interpolate between node contents it can be set a ***fade time*** that will determine a transition between a new and a previous setting.

Node Proxy

Using ***set*** and ***map*** we can connect node proxies parameters. Using ***xset*** and ***xmap*** we can do so using a transition of values.

Node, pattern and task proxies all support a ***quantization*** setting to specify when changes will be applied. Furthermore we can set the ***onset*** and ***condition*** of transition.

For all NodeProxies, parameters can be set before they are used.

For easy access of synthesis parameters one can use ***TdefGui***, ***PdefGui*** and ***NdefGui*** for testable guis. ***ProxyMixer*** can also be used to mix processes in an environment.

Task Proxy

Creating sequences of actions that wait for a determined amount of time tasks and their JITLib versions ***Tdef*** and ***TaskProxy*** can be used.

Using the ***.embed*** and ***.fork*** methods, subtasks ban either be run in sequence or in parallel.

Patterns

Patterns also have their JITLib variants. For object streams such as for numerical values, ***PatternProxy*** and ***Pdefn*** should be used. For event streams such as pbind types, ***EventPatternProxy*** and ***Pdef*** should be used.

Pdef behaves in a similar way to node proxies so that its containing sub patterns that can also be modified during run time.

Pbindef allows keys to replaced for a playing pattern.

liveCoding_101_Weekender

LIVE-CODING BEGINNER-LEVEL INTERMEDIATE-LEVEL WEEKENDER WORKSHOP SHOWCASE

Tickets

Join us a weekend full of workshops on live coding music and visuals, and a from-scratch session!

On Saturday you will learn programming visuals by Niki Scheijen (Nikilia) with the Hydra Video Synth, and you will also learn programming music by Wilbert Vogel (Lil' Bleep) with the Mercury playground. **No experience required, just curiosity!**

In the evening we'll have a From-Scratch session together with the Netherlands Coding Live (NL_CL) community, where you can enjoy short 9-minute improvised performances by others, or if you already feel like it you can join yourself!

https://creativecodingutrecht.nl/en/calendar/live-coding-101

Exercises

```
(
NF(\iop, {|freq=78, mul=1.0, add=0.0|
    var noise = LFNoise1.ar(0.001).range(freq, freq + (freq * 0.1));
    var osc = SinOsc.ar([noise, noise * 1.04, noise * 1.02, noise * 1.08],0,0.2);
    var out = DFM1.ar(osc,freq*4,SinOsc.kr(0.01).range(0.92,1.05),1,0,0.005,0.7);
    HPF.ar(out, 40)
}).play;
)

(
NF(\dsc, {|freq = 1080|
    HPF.ar(
        BBandStop.ar(Saw.ar(LFNoise1.ar([19,12]).range(freq,freq*2), 0.2).excess(
        SinOsc.ar( [freq + 6, freq + 4, freq + 2, freq + 8])),
        LFNoise1.ar([12,14,10]).range(100,900),
        SinOsc.ar(20).range(9,11)
    ), 80)
    ;
}).play;
)
```

```
var <>pindex, <>cindex;

initialize {
    if(pindex.isNil, { pindex = 1000 });
    if(cindex.isNil, { cindex = 2000 });
}

clearProcessSlots {
    pindex = 1000;
    (this.pindex - 1000).do{|i| this[this.pindex+i] = nil; }
}

clearOrInit {|clear=true|
    if(clear == true, { this.clearProcessSlots() }, { this.initialize() });
}

transform {|process, index|
    if(index.isNil && pindex.isNil, {
        this.initialize();
    });

    pindex = pindex + 1;
    this[pindex] = \filter -> process;
}

control {|process, index|
    var i = index;

    if(i.isNil, {
        this.initialize();
        cindex = cindex + 1;
        i = cindex;
    });

    this[i] = \pset -> process;
}
```

```
(
NF(\depfm, {|freqMin=5, freqMax=20, mul=20, add=80, rate=0.5, modFreq=2100, index=0.3, amp=0.2|
    var trig, seq, freq;
    trig = Dust.kr(rate);
    seq = Diwhite(freqMin, freqMax, inf).midicps;
    freq = Demand.kr(trig, 0, seq);
    HPF.ar(PMOsc.ar(LFCub.kr([freq, freq/2, freq/3, freq/4], 0, mul, add),
    LFNoise1.ar(0.3).range(modFreq,modFreq*2), index) * amp, 50)
}).play;
)
```

Exercises

1. Employee a Tdef to create a short sequence of at least two different **Ndefs** that can the be changed while the **Tdef** runs.

2. Create a source and effect chain where the source is processed by the effect using **Ndefs**.

3. Create a modulated synth that contains two modulation sources that can be set using **proxies** (LFOs, Envelopes etc.)

4. Create a small pool of live coding snippets that could be triggered when creating a piece of performance.The pool should contain different sources for event, modulation and sound generation.